

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EduCore Student Portal</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <header>
    <h1>EduCore Student Portal</h1>
  </header>

  <main>
    <h2>Courses</h2>

    <input id="search-courses" type="text" placeholder="Search courses...">
    <button id="sort-btn" type="button">Sort by Credits</button>

    <div class="course-grid"></div>

    <p id="total-credits"></p>

    <div id="selected-course"></div>
  </main>

  <script type="module" src="app.js"></script>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body { font-family: Arial, sans-serif; font-size: 16px; background: #f5f5f5; color: #333; }

header { background: #1e3a5f; color: #fff; padding: 16px 24px; }

main { max-width: 960px; margin: 24px auto; padding: 0 16px; }

h2 { margin-bottom: 12px; }
```

```
#search-courses { padding: 8px; width: 200px; margin-bottom: 8px; margin-right: 8px; }
```

```
button { padding: 8px 16px; cursor: pointer; margin-bottom: 16px; }
```

```
.course-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
  gap: 16px;  
  margin-bottom: 16px;  
}
```

```
.course-card {  
  background: #fff;  
  border: 1px solid #ddd;  
  border-radius: 4px;  
  padding: 16px;  
  cursor: pointer;  
}
```

```
.course-card:hover { background: #f0f4ff; }
```

```
#total-credits { margin-top: 8px; font-weight: bold; }
```

```
#selected-course { margin-top: 12px; color: #1e3a5f; font-weight: bold; }
```

app.js

```
// Task 1 — Step 30: import courses using ES6 import  
import { courses } from './data.js';
```

```
// Step 30: destructuring in a loop  
courses.forEach(course => {  
  const { name, credits } = course;  
  console.log(name, credits);  
});
```

```
// Step 31: map — formatted strings  
const formatted = courses.map(({ code, name, credits }) =>  
  `${code} — ${name} (${credits} credits)`  
);  
console.log('Formatted:', formatted);
```

```
// Step 32: filter — credits >= 4
```

```

const highCredit = courses.filter(course => course.credits >= 4);
console.log('Courses with 4+ credits:', highCredit.length);

// Step 33: reduce — total credits
const totalCredits = courses.reduce((sum, course) => sum + course.credits, 0);
console.log('Total credits:', totalCredits);

// Task 2 — Step 36: select the grid container
const grid = document.querySelector('.course-grid');
const totalEl = document.getElementById('total-credits');
const selectedEl = document.getElementById('selected-course');

// Step 37–38: render course cards using createElement + template literal
function renderCourses(list) {
  // clear before re-render to avoid duplicates
  grid.innerHTML = '';

  // use DocumentFragment for batch append (better performance)
  const fragment = document.createDocumentFragment();

  list.forEach(course => {
    const card = document.createElement('article');
    card.className = 'course-card';
    card.dataset.id = course.id;
    card.innerHTML = `
      <h3>${course.code}</h3>
      <p>${course.name}</p>
      <p>${course.credits} Credits</p>
    `;
    fragment.appendChild(card);
  });

  grid.appendChild(fragment);

  // Step 39: update total credits
  const total = list.reduce((sum, c) => sum + c.credits, 0);
  totalEl.textContent = `Total Credits: ${total}`;
}

// initial render
renderCourses(courses);

```

```

// Task 3 — Step 41: search filter on input event
document.getElementById('search-courses').addEventListener('input', function () {
  const query = this.value.toLowerCase();
  const filtered = courses.filter(c => c.name.toLowerCase().includes(query));
  renderCourses(filtered);
});

// Step 42: sort by credits descending on button click
document.getElementById('sort-btn').addEventListener('click', () => {
  const sorted = [...courses].sort((a, b) => b.credits - a.credits);
  renderCourses(sorted);
});

// Step 44: event delegation — single listener on grid container
grid.addEventListener('click', (event) => {
  // Step 43+44: detect which card was clicked
  const card = event.target.closest('.course-card');
  if (!card) return;

  const id = Number(card.dataset.id);
  const course = courses.find(c => c.id === id);

  // Step 43: display course details
  selectedEl.textContent = `Selected: ${course.name} | Grade: ${course.grade}`;
});

```

data.js

```

// Task 1 — Step 29: export course data
export const courses = [
  { id: 1, code: 'CS101', name: 'Intro to Programming', credits: 4, grade: 'A' },
  { id: 2, code: 'CS201', name: 'Data Structures', credits: 4, grade: 'B' },
  { id: 3, code: 'MA101', name: 'Calculus I', credits: 3, grade: 'A' },
  { id: 4, code: 'EC101', name: 'Circuit Theory', credits: 3, grade: 'B' },
  { id: 5, code: 'ME101', name: 'Engineering Mechanics', credits: 3, grade: 'C' },
];

```

OUTPUT:

EduCore Student Portal

Courses

Sort by Credits

CS101

Intro to Programming

4 Credits

CS201

Data Structures

4 Credits

MA101

Calculus I

3 Credits

EC101

Circuit Theory

3 Credits

ME101

Engineering Mechanics

EduCore Student Portal

Courses

Sort by Credits

CS101

Intro to Programming

4 Credits

Total Credits: 4